

INVESTIGACIÓN PREVIA

Autenticación, autorización, LocalStorage, DOM y buenas prácticas de prompts con IA

Base conceptual para un sistema de autenticación por roles
utilizando HTML, CSS, JavaScript y LocalStorage

Estudiante

Curso

Docente

Fecha

6 de agosto de 2026

Documento elaborado con fines académicos

Contenido

Introducción.....	3
1. ¿Qué es la autenticación?.....	3
2. Diferencia entre autenticación y autorización.....	3
3. Importancia de implementar un inicio de sesión.....	4
4. Ventajas de un sistema administrativo.....	5
5. ¿Qué es LocalStorage?.....	5
6. ¿Qué es el DOM?.....	6
7. Buenas prácticas para construir prompts efectivos.....	6
Aplicación de los conceptos en el mini proyecto.....	8
Conclusión.....	8
Referencias.....	8

Propósito del documento: Investigar los conceptos solicitados antes de construir el proceso de autenticación por roles con LocalStorage. Además, se explica cómo se relacionan entre sí dentro de una aplicación web.

Introducción

Antes de desarrollar una aplicación web con inicio de sesión, es necesario comprender qué significa identificar a una persona, verificar sus credenciales y controlar las acciones que puede realizar. También es importante conocer las herramientas del navegador que permiten guardar información y modificar la interfaz de una página.

Esta investigación explica siete conceptos fundamentales para el anteproyecto: autenticación, autorización, inicio de sesión, sistemas administrativos, LocalStorage, DOM e ingeniería de prompts. La finalidad es construir una base teórica clara y relacionarla con un sistema sencillo de acceso por roles.

1. ¿Qué es la autenticación?

La autenticación es el proceso mediante el cual un sistema comprueba que una persona, aplicación o dispositivo realmente es quien afirma ser. OWASP la define como la verificación de una identidad utilizando uno o más autenticadores, por ejemplo una contraseña, una huella digital o un token de seguridad [1].

En una aplicación web, la autenticación normalmente ocurre de la siguiente manera:

1. El usuario indica quién es, por ejemplo, mediante un nombre de usuario o correo electrónico.
2. El usuario presenta una evidencia, como una contraseña.
3. El sistema compara esa evidencia con la información registrada.
4. Si la comparación es correcta, el sistema reconoce al usuario y crea una sesión.

Los métodos de autenticación se pueden agrupar en tres factores principales:

Factor	Significado	Ejemplo
Algo que sabe	Información conocida por el usuario	Contraseña o PIN
Algo que tiene	Objeto que posee el usuario	Teléfono, tarjeta o token
Algo que es	Característica física o biométrica	Huella o reconocimiento facial

Ejemplo: Cuando una persona escribe su correo y contraseña para entrar a un panel administrativo, el sistema está realizando autenticación.

2. ¿Cuál es la diferencia entre autenticación y autorización?

Aunque ambos conceptos forman parte de la seguridad de acceso, responden preguntas diferentes. La autenticación responde “¿quién eres?”, mientras que la autorización responde “¿qué tienes permitido hacer?”. La autorización se aplica después de conocer la identidad del usuario y debe ajustarse a las reglas del negocio [2].

Aspecto	Autenticación	Autorización
Pregunta principal	¿Quién es el usuario?	¿Qué puede hacer?
Momento	Sucede al iniciar sesión	Sucede al acceder a funciones o datos
Datos utilizados	Credenciales o factores de identidad	Rol, permisos y políticas
Ejemplo	Validar correo y contraseña	Permitir que solo el administrador elimine usuarios

Ejemplo aplicado: dos personas pueden autenticarse correctamente en el mismo sistema, pero recibir opciones distintas. Un administrador puede crear, modificar y eliminar registros; un empleado puede únicamente consultar o registrar información. Ambos están autenticados, pero tienen autorizaciones diferentes.

3. ¿Por qué es importante implementar un inicio de sesión en un sistema?

Un inicio de sesión es la interfaz mediante la cual el usuario presenta sus credenciales para autenticarse. Su importancia no se limita a mostrar un formulario: permite que el sistema sepa quién está utilizando la aplicación y aplique controles de acceso. OWASP señala que la confirmación de identidad, la autenticación y la gestión de sesión son esenciales para proteger una aplicación contra ataques relacionados con credenciales [3].

- Protege información y funciones que no deben estar disponibles para cualquier visitante.
- Permite aplicar roles y permisos diferentes según el tipo de usuario.
- Facilita la personalización, por ejemplo mostrar el nombre, el rol o las opciones correspondientes.
- Ayuda a mantener trazabilidad, porque las acciones pueden relacionarse con una cuenta concreta.
- Evita que personas no autorizadas ingresen directamente a módulos administrativos.
- Permite cerrar la sesión y limpiar el estado del usuario cuando termina su trabajo.

Importante: El formulario de login por sí solo no brinda seguridad. También se debe validar la información, controlar la sesión y comprobar los permisos antes de cada acción sensible.

4. ¿Qué ventajas ofrece un sistema administrativo?

Un sistema administrativo es una aplicación que ayuda a registrar, organizar y controlar procesos de una organización. Puede administrar usuarios, clientes, productos, ventas, inventario, citas, documentos u otros recursos, según las necesidades del negocio.

Las plataformas de gestión empresarial suelen integrar procesos y datos en un mismo lugar. Microsoft explica que la centralización, la automatización y el acceso a información en tiempo real pueden mejorar la eficiencia y apoyar la toma de decisiones [4].

Centralización de información: Los datos se encuentran en una sola plataforma, lo cual reduce archivos dispersos y duplicados.

Automatización: Tareas repetitivas, como cálculos, validaciones o generación de reportes, pueden ejecutarse con menos trabajo manual.

Control por roles: Cada usuario visualiza únicamente las opciones necesarias para sus responsabilidades.

Rapidez de consulta: La información puede buscarse, filtrarse y actualizarse con mayor facilidad.

Reportes y decisiones: Los responsables pueden consultar indicadores y utilizar datos actualizados para decidir.

Trazabilidad: El sistema puede registrar quién realizó una acción y cuándo ocurrió.

Reducción de errores: Las validaciones y los formularios estandarizados disminuyen inconsistencias en los datos.

Escalabilidad: El sistema puede crecer mediante nuevos módulos, usuarios o funciones.

Relación con el anteproyecto: un panel administrativo basado en roles puede mostrar un menú completo al administrador y un menú limitado al usuario regular. De esta manera se demuestra la diferencia entre autenticación y autorización.

5. ¿Qué es LocalStorage?

LocalStorage es una característica del navegador que permite guardar información en forma de pares clave-valor. Los datos pertenecen al origen de la página y normalmente permanecen disponibles después de cerrar y volver a abrir el navegador [5].

Los valores se almacenan como texto. Cuando se desea guardar un arreglo u objeto de JavaScript, se convierte a JSON con `JSON.stringify()` y se recupera con `JSON.parse()`.

Método	Función
<code>localStorage.setItem(clave, valor)</code>	Guarda o actualiza un dato.
<code>localStorage.getItem(clave)</code>	Obtiene un dato guardado.
<code>localStorage.removeItem(clave)</code>	Elimina una clave específica.
<code>localStorage.clear()</code>	Elimina todos los datos del origen.

Ejemplo básico:

```
const usuario = { nombre: 'Ana', rol: 'admin' };

localStorage.setItem('usuarioActual', JSON.stringify(usuario));

const guardado = JSON.parse(localStorage.getItem('usuarioActual'));
console.log(guardado.rol); // admin
```

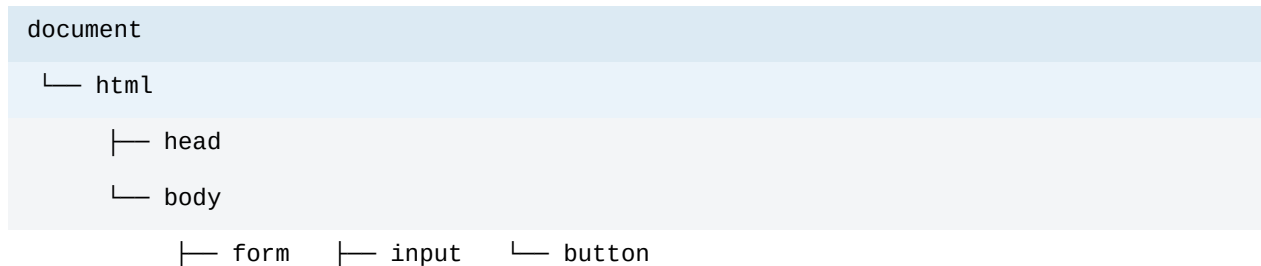
Limitación de seguridad: LocalStorage es útil para prácticas académicas y datos no sensibles, pero no debe considerarse una autenticación segura para producción. El usuario puede modificar su contenido desde las herramientas del navegador, y un ataque XSS puede leer los datos almacenados. OWASP recomienda no guardar contraseñas, identificadores de sesión ni tokens de autenticación en LocalStorage [6].

Uso recomendado en este mini proyecto: guardar usuarios de prueba, el rol seleccionado y el estado de una sesión simulada. Para un sistema real se necesitaría un servidor, una base de datos, contraseñas protegidas y sesiones seguras.

6. ¿Qué es el DOM?

DOM significa Document Object Model o Modelo de Objetos del Documento. Es una interfaz que representa la estructura de una página HTML en memoria como un árbol de nodos. Gracias al DOM, JavaScript puede acceder a los elementos, cambiar el contenido, modificar estilos, crear componentes y responder a eventos [7].

Representación simplificada de un árbol DOM:



Algunas operaciones frecuentes con el DOM son:

- Seleccionar elementos mediante `document.getElementById()` o `document.querySelector()`.
- Leer o cambiar texto con `textContent`.
- Cambiar clases de CSS con `classList`.
- Crear elementos con `document.createElement()`.
- Escuchar eventos como click, submit o change mediante `addEventListener()`.

Ejemplo aplicado al formulario de inicio de sesión:

```

const formulario = document.querySelector('#loginForm');
const mensaje = document.querySelector('#mensaje');

formulario.addEventListener('submit', (evento) => {
  evento.preventDefault();
  mensaje.textContent = 'Validando credenciales...';
});
  
```

Relación con el proyecto: el DOM permite capturar los datos del formulario, mostrar mensajes de error, ocultar o mostrar secciones según el rol y actualizar el panel sin tener que modificar manualmente el archivo HTML cada vez.

7. Buenas prácticas para construir prompts efectivos utilizando IA

Un prompt es la instrucción o entrada que se proporciona a un modelo de inteligencia artificial. La ingeniería de prompts consiste en diseñar y mejorar esas instrucciones para orientar la respuesta del modelo. OpenAI recomienda escribir solicitudes claras y específicas, aportar contexto suficiente y refinar el prompt de manera iterativa después de revisar la respuesta [8].

Definir el objetivo: Explicar exactamente qué resultado se necesita y para qué se utilizará.

Asignar un rol: Indicar desde qué perspectiva debe responder la IA, por ejemplo: “Actúa como desarrollador web principiante”.

Proporcionar contexto: Describir el proyecto, los usuarios, las reglas y las tecnologías permitidas.

Incluir restricciones: Señalar lo que no debe utilizarse, la extensión máxima, el nivel de dificultad o las normas del profesor.

Pedir un formato: Solicitar una tabla, JSON, pasos numerados, código por archivos o una explicación en secciones.

Dar ejemplos: Mostrar una entrada y una salida esperada cuando el formato pueda ser ambiguo.

Dividir tareas complejas: Solicitar primero la estructura, luego una funcionalidad, después las pruebas y finalmente la revisión.

Revisar e iterar: Comprobar la respuesta, identificar errores y corregir el prompt en lugar de aceptar la primera salida.

Verificar la información: La IA puede equivocarse o inventar datos; el usuario debe probar el código y revisar las fuentes.

Evitar datos sensibles: No incluir contraseñas reales, tokens, información privada ni datos confidenciales en el prompt.

Comparación de prompts:

Prompt poco efectivo	Prompt mejorado
"Hazme un login."	"Actúa como estudiante principiante de desarrollo web. Crea un formulario de inicio de sesión con HTML, CSS y JavaScript puro. Los usuarios deben almacenarse en LocalStorage y existirán los roles admin y empleado. No uses librerías. Devuelve el código separado por archivos y explica cómo probarlo. Aclara que LocalStorage se utiliza únicamente con fines académicos."

Ciclo recomendado: Planear lo que se necesita → escribir el prompt con contexto → revisar la respuesta → probar el resultado → corregir una cosa a la vez → repetir hasta obtener una solución confiable.

Aplicación de los conceptos en el mini proyecto

Los temas investigados se pueden integrar en un sistema académico de autenticación por roles. El flujo general podría ser el siguiente:

1. Al iniciar la aplicación, JavaScript comprueba si existe una lista de usuarios de prueba en LocalStorage. Si no existe, crea cuentas iniciales con roles diferentes.
2. El DOM permite leer el correo y la contraseña escritos en el formulario.
3. La aplicación busca una coincidencia dentro de los usuarios almacenados. Esta comparación representa una autenticación simulada.
4. Si las credenciales son válidas, se guarda un objeto de usuario actual con su nombre y rol.
5. La interfaz consulta el rol y muestra las opciones permitidas. Esto representa una autorización básica.
6. El panel administrativo puede permitir más acciones al rol admin y limitar al rol empleado.
7. Al cerrar sesión, se elimina el usuario actual y se regresa al formulario de login.

Etapas	Concepto principal	Ejemplo
Formulario	DOM	Capturar usuario y contraseña
Validación	Autenticación	Comprobar credenciales
Permisos	Autorización	Mostrar funciones según el rol
Persistencia	LocalStorage	Guardar usuarios y sesión simulada
Mejora del desarrollo	Prompts efectivos	Solicitar código con reglas y formato claros

Alcance académico: Este diseño demuestra los conceptos solicitados y puede servir para un prototipo local. No debe publicarse como sistema de seguridad real, ya que toda la validación ocurre en el navegador y puede ser modificada por el usuario.

Conclusión

La autenticación permite comprobar la identidad de un usuario, mientras que la autorización determina las acciones que puede realizar. Un inicio de sesión integra estos conceptos y ayuda a proteger las funciones de un sistema administrativo. A su vez, los sistemas administrativos ofrecen ventajas como centralización, automatización, control por roles, trazabilidad y generación de información para tomar decisiones.

LocalStorage y el DOM son herramientas útiles para desarrollar un prototipo web sin servidor. LocalStorage conserva datos en el navegador y el DOM permite interactuar con la interfaz. Sin embargo, LocalStorage no debe emplearse como mecanismo de seguridad para credenciales o sesiones reales.

Finalmente, el uso de prompts claros, específicos y con suficiente contexto facilita obtener apoyo de la inteligencia artificial durante el desarrollo. La respuesta de la IA siempre debe revisarse, probarse y corregirse. Con esta base conceptual, es posible construir el mini proyecto solicitado de forma ordenada y comprender qué representa cada parte del código.

Referencias

- [1] OWASP Cheat Sheet Series. (s. f.). Authentication Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [2] OWASP Cheat Sheet Series. (s. f.). Authorization Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

- [3] OWASP Foundation. (2021). A07:2021 - Identification and Authentication Failures.
https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/
- [4] Microsoft Dynamics 365. (s. f.). What is ERP?.
<https://www.microsoft.com/en/dynamics-365/resources/what-is-erp>
- [5] MDN Web Docs. (2025). Window: localStorage property.
<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [6] OWASP Cheat Sheet Series. (s. f.). HTML5 Security Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
- [7] MDN Web Docs. (2026). Modelo de Objetos del Documento (DOM).
https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model
- [8] OpenAI Help Center. (2026). Mejores prácticas de ingeniería de prompts para ChatGPT.
<https://help.openai.com/es-419/articles/10032626-prompt-engineering-best-practices-for-chatgpt>

Nota metodológica: Las fuentes se consultaron para elaborar una síntesis académica. Las explicaciones y ejemplos fueron redactados en lenguaje propio para relacionarlos con el anteproyecto solicitado.